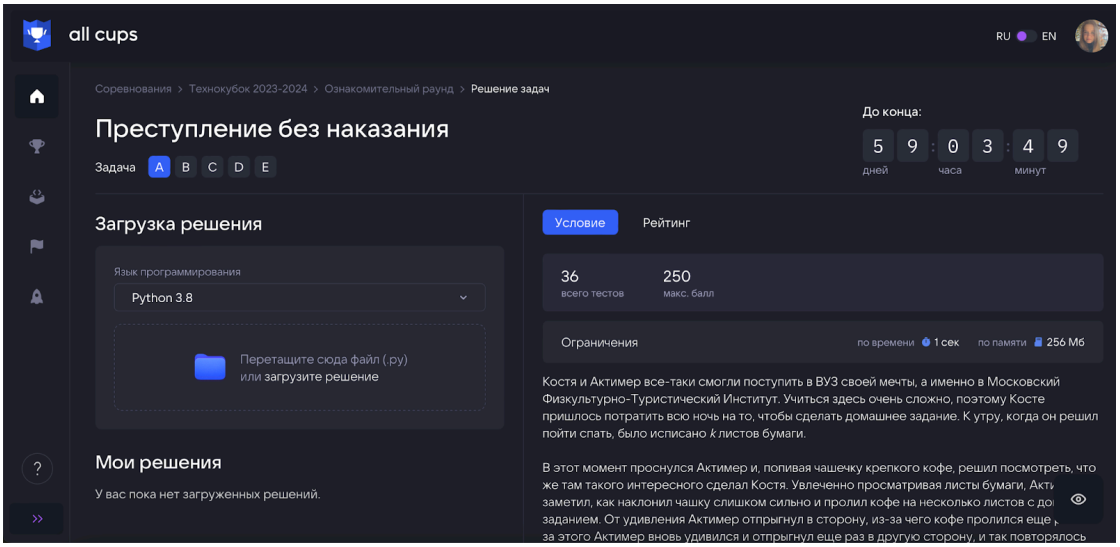
Инструкция по решению задач в курсе с интегрированной системой All Cups

После того, как ты был зарегистрирован на учебную программу с интегрированной системой All Cups, проверь, что с момента поступления тебе уведомления о зачислении на почту прошло минимум 15 минут.

1. Чтобы приступить к решению задачи, необходимо нажать на кнопку “Решать задачи”. 

После этого откроется рабочее пространство – интерфейс, в котором ты сможешь увидеть условия, формат входных и выходных данных, загрузить решения задач.



2. Тебе необходимо (согласно условиям задачи) подготовить решение локально на компьютере и загрузить его в систему проверки или написать решение сразу в системе.
3. Помимо условий самого задания к твоему решению предъявляются некоторые требования со стороны системы проверки:

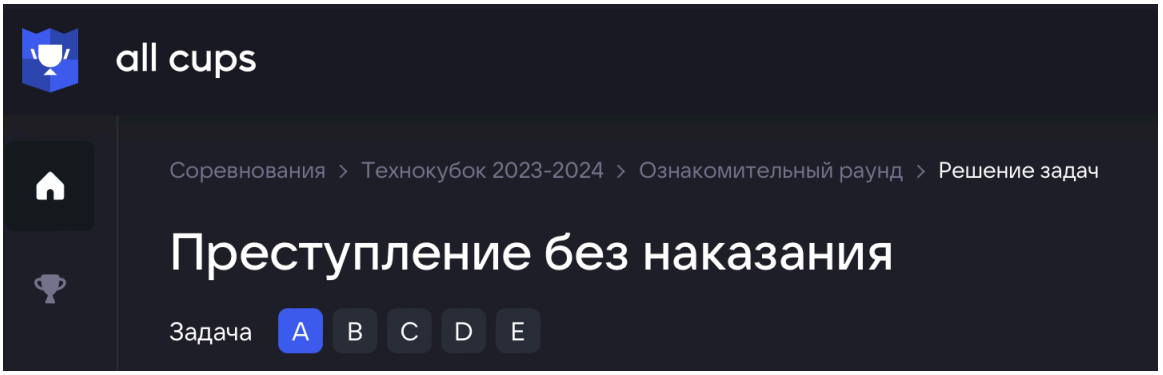
- Твоя программа получает стандартный потоковой ввод данных (stdin).
- Твоя программа вернет ответы в стандартном потоковом выводе данных (stdout).

4. По результатам проверки ты можешь получить определенный код для своего решения:

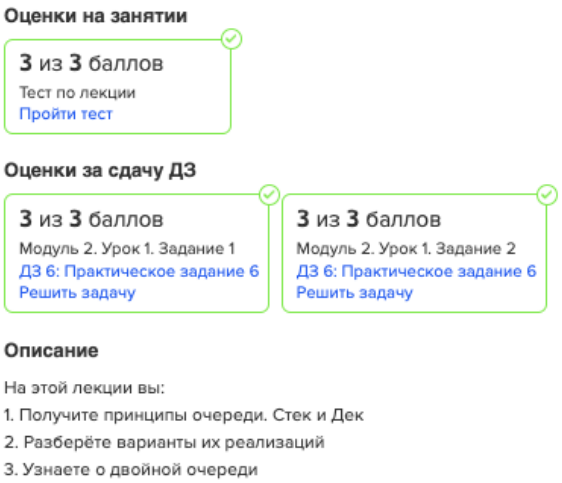
- **OK (Correct answer)** – Правильный ответ. Тест с таким ответом будет считаться успешно пройденным. Для того, чтобы задача считалась полностью решенной, все тесты должны быть успешно пройдены.
- **WA (Wrong answer)** – Неверный ответ. Ответ программы не совпадает с эталонным ответом, или не проходит проверку “чекером” - проверяющей программой, анализирующей задачи, в которых возможно несколько вариантов правильных ответов. В некоторых особых случаях (зависит от задачи) в тексте сообщения об ошибке могут содержаться подробные причины ошибки.
- **PE (Presentation error)** – Не удалось проверить правильность ответа, поскольку формат вывода неверен. Например, количество строк не совпадает с количеством строк в ответе, или строка содержит меньшее/большее количество элементов, чем ожидается по условиям задачи.
- **RE (Runtime error)** – Ошибка времени выполнения. Например, деление на 0, переполнение стека, и т. п.
- **ML (Memory limit exceeded)** – Превышено ограничение по памяти. Ограничение по памяти зависит от задачи и указано в ее описании.
- **TL (Time limit exceeded)** – Превышено время выполнения программы. Ограничение по времени зависит от задачи и указано в ее описании.
- **IL (Idleness limit exceeded)** – Превышено время бездействия. Если по каким-то причинам от программы в течение заданного временного лимита не зафиксировано никакой активности (то есть, программа не использует процессорное время).
- **SV (Security violation)** – Программа выполнила небезопасные операции. Любые попытки взлома проверяющей системы, нарушения правил, выполнение подозрительных действий, не связанных с условиями задачи.

- **CR (non-zero Code Returned)** – Программа завершилась с ненулевым кодом возврата. Это может происходить по причине ошибки времени выполнения или, если код возврата, отличный от 0, указан явно: например, return 1;.
- **SE (Server Error)** – Непредвиденная ошибка в проверяющей системе. Сообщите об этом факте организатору. Решения с такими ошибками будут пересчитаны после выяснения причин и устранения неполадок.

5. Для перехода к следующей задаче выбери следующее буквенное обозначение в раунде



или вернись в систему https://education.vk.company/ и выбери следующее задание



Вопросы и ответы

Как я пойму, что ответ сохранился?

После загрузки решения в систему ответ сохраняется автоматически

У меня не прогружается задание, что делать?

Убедись, что используешь Google Chrome или Safari, у тебя отключен VPN и блокировщик рекламы. Обнови страницу.

Как я пойму, сколько времени у меня осталось?

Внутри рабочего пространства в верхнем правом углу будет таймер, отсчитывающий время до конца раунда.

Как сдавать решения на проверку?

На проверку отправляется исходный текст программы на одном из допустимых языков программирования. Обычно процесс проверки занимает не более 1-3 минут, однако иногда по техническим причинам может проходить и большее время. Все решения, посланные на проверку, рано или поздно будут проверены.

После проверки моего решения появилась ошибка. Почему?

- **OK (Correct answer)** – Правильный ответ. Тест с таким ответом будет считаться успешно пройденным. Для того, чтобы задача считалась полностью решенной, все тесты должны быть успешно пройдены.
- **WA (Wrong answer)** – Неверный ответ. Ответ программы не совпадает с эталонным ответом, или не проходит проверку “чекером” - проверяющей программой, анализирующей задачи, в которых возможно несколько вариантов правильных ответов. В некоторых особых случаях (зависит от задачи) в тексте сообщения об ошибке могут содержаться подробные причины ошибки.
- **PE (Presentation error)** – Не удалось проверить правильность ответа, поскольку формат вывода неверен. Например, количество строк не совпадает с количеством строк в ответе, или строка содержит меньшее/большее количество элементов, чем ожидается по условиям задачи.
- **RE (Runtime error)** – Ошибка времени выполнения. Например, деление на 0, переполнение стека, и т. п.

- **ML (Memory limit exceeded)** – Превышено ограничение по памяти. Ограничение по памяти зависит от задачи и указано в ее описании.
- **TL (Time limit exceeded)** – Превышено время выполнения программы. Ограничение по времени зависит от задачи и указано в ее описании.
- **IL (Idleness limit exceeded)** – Превышено время бездействия. Если по каким-то причинам от программы в течение заданного временного лимита не зафиксировано никакой активности (то есть, программа не использует процессорное время).
- **SV (Security violation)** – Программа выполнила небезопасные операции. Любые попытки взлома проверяющей системы, нарушения правил, выполнение подозрительных действий, не связанных с условиями задачи.
- **CR (non-zero Code Returned)** – Программа завершилась с ненулевым кодом возврата. Это может происходить по причине ошибки времени выполнения или если код возврата, отличный от 0, указан явно: например, return 1;.
- **SE (Server Error)** – Непредвиденная ошибка в проверяющей системе. Сообщите об этом факте организатору. Решения с такими ошибками будут пересчитаны после выяснения причин и устранения неполадок.

Почему программа не проходит тесты из условия, хотя на моем компьютере работает правильно?

Как правило, наиболее вероятные причины этого следующие:

- Программа читает строку при помощи стандартных функций языка программирования, например, getline в C/C++, input в Python, при этом при сдаче программы желаемый тип концов строк установлен в CRLF. В этом случае программа при тестировании может считать лишний байт с кодом 0D, который в операционной системе Linux не будет восприниматься, как служебный символ конца строки. Пересдайте программу, не устанавливая параметр “желаемый тип концов строк” при отправке решения.
- Наоборот, программа читает данные посимвольно, концы строк анализируются непосредственно в программе, сравнивая значения считанных символов с кодами 0D и 0A. При такой технологии чтения входных данных необходимо при сдаче программы указать желаемый тип концов строк CRLF.

- Программа читает данные из файла, при этом данные считываются посимвольно до конца файла или как строки. В тестирующей системе последняя строка входных данных завершается символом конца строки, в то время как учащийся при отладке программы на своем компьютере использует входные файлы, не содержащие символа конца строки в конце файла. Необходимо добавить в конец тестового файла символ конца строки и еще раз проверить программу на своем компьютере.
- Программа содержит ошибку, из-за которой программа может по-разному выполняться на разных компьютерах. Как правило, такой ошибкой является отсутствие инициализации переменной или выход за границы массива.

Техническая поддержка

Если у тебя возникла какая-то техническая сложность и решение проблемы на этой странице не найдено, напиши обращение на info@education.vk.company

Обязательно укажи:

– свой email участника;

– описание проблемы;

– скриншоты / видеозапись проблемы (при необходимости).

Работа с системой проверки решений

Для решения практических задач мы рекомендуем вам работать с версиями языков программирования и пакетами, перечисленными ниже.

Данные пакеты установлены на стороне системы проверки.

Требования к решению

Ваша программа получает стандартный потоковой ввод данных (stdin).

Мы ожидаем, что ваша программа вернет ответы в стандартном потоковом выводе данных (stdout).

Ограничения на решения

Перечисленные ниже ограничения применяются ко всем задачам независимо от языка программирования (кроме SQL) по умолчанию. При этом, в каждой задаче будут добавлены свои лимиты. Смотрите условия.

Настройки по умолчанию

```
{
  "execution_time_s": 2,
  "cpu_execution_time_s": 2,
  "peak_ram_used_mb": 128,
  "peak_swap_used_mb": 128,
  "compare_mode": "case_insensitive"
}
```

- execution_time_s: максимальное время выполнения в секундах;
- cpu_execution_time_s: максимальное время потребления ресурса CPU в секундах;
- peak_ram_used_mb: максимальное пиковое потребление оперативной памяти в мегабайтах;
- peak_swap_used_mb: максимальное пиковое потребление свопа в мегабайтах;
- case_insensitive: не учитывается регистр в ответе.

Go

Операционная система: Ubuntu 20.04

Версия языка Go: 1.14.6

Команды установки:

```
apt-get update -y && apt-get install --no-install-recommends -y -q
curl build-essential ca-certificates git mercurial bzr
mkdir /goroot && curl
<https://dl.google.com/go/go1.14.6.linux-amd64.tar.gz> | tar xvzf
- -C /goroot --strip-components=1
mkdir /gopath
go install -buildmode=shared std
```

Компиляция:

```
go build -linkshared -o $COMPILED_FILE_PATH
$SOLUTION_CODE_PATH/$SOLUTION_CODE_ENTRYPOINT 2>&1 >/dev/null
```

Запуск:

```
/lib64/ld-linux-x86-64.so.2
$SOLUTION_CODE_PATH/$SOLUTION_CODE_ENTRYPOINT
```

Обращаем ваше внимание, что вместо переменных вам необходимо вставить свои параметры для компиляции и запуска.

Пример программы:

```
None
package main

import (
    "fmt"
)

func main() {
    fmt.Scanf("%d", &str)
    fmt.Printf("%d\\n", str)
}
```

Python

Операционная система: Ubuntu 20.04

Версия языка Python: 3.8

Команды установки:

```
apt-get update && \\
     apt-get install -y python3.8 python3-pip
```

Запуск:

```
python3.8 -u $SOLUTION_CODE_PATH/$SOLUTION_CODE_ENTRYPOINT
```

Обращаем ваше внимание, что вместо переменных вам необходимо вставить свои параметры для компиляции и запуска.

Пример программы:

```
None
if __name__ == "__main__":
     input_str = input()
     print(input_str)
```

Java

Операционная система: Ubuntu 20.04

Версия языка Java: 11

Команды установки:

```
apt-get update -y && \\
     apt-get install -y software-properties-common

add-apt-repository -y ppa:openjdk-r/ppa && \\
     apt-get update -y && \\
     apt-get install -y openjdk-11-jdk && \\
     apt-get install -y maven

mkdir -p src/main/java && \\
     mvn dependency:go-offline && \\
     mvn package && \\
     rm -rf compiled.jar target/classes/
```

Примечание: возможно потребуется объявление переменной MAVEN_OPTS="-Djavax.net.ssl.trustStore=/etc/ssl/certs/java/cacerts -Djavax.net.ssl.trustStorePassword=changeit --add-opens java.base/java.lang=ALL-UNNAMED"

Компиляция:

```
mvn package -q -Dstyle.color=never
```

Запуск:

```
java -jar ${MOUNT_POINT}
```

Обращаем ваше внимание, что вместо переменных вам необходимо вставить свои параметры для компиляции и запуска.

Пример программы:

None

```
import java.util.Scanner;

public class Main {
     public static void main(String[] args) {
     Scanner in = new Scanner(System.in);
     System.out.print(in);
     }
}
```

PHP

Операционная система: Ubuntu 20.04

Версия языка PHP: 7.4

Команды установки:

```
apt-get update -y && \\
     apt-get install -y software-properties-common
DEBIAN_FRONTEND=noninteractive LC_ALL=en_US.UTF-8
add-apt-repository -y ppa:ondrej/php && \\
     apt-get update && \\
     apt-get -y install php7.4 php7.4-cli php7.4-common
php7.4-json php7.4-mbstring
```

Запуск:

```
usr/bin/php $SOLUTION_CODE_PATH/$SOLUTION_CODE_ENTRYPOINT
```

Обращаем ваше внимание, что вместо переменных вам необходимо вставить свои параметры для компиляции и запуска.

Пример программы:

```
None
<?php
$var1 = readline();
echo $var1;
?>
```

C++

Операционная система: Ubuntu 20.04

Версия языка C++: 17

Команды установки:

```
apt-get update -y && \\
     apt-get install -y software-properties-common
add-apt-repository ppa:ubuntu-toolchain-r/test -y && \\
     apt-get update -y && \\
     apt-get install -y g++-9 make cmake
```

Компиляция:

```
make 2>&1 >/dev/null
```

Запуск:

```
/lib64/ld-linux-x86-64.so.2 $MOUNT_POINT
```

Обращаем ваше внимание, что вместо переменных вам необходимо вставить свои параметры для компиляции и запуска.

Пример программы:

```
None
#include <iostream>
int main() {
     int n;
     std::cin >> n;
  std::cout<<i;
     return 0;
}
```

C#

Операционная система: Ubuntu 20.04

Версия языка C#: 6.12.0 Stable

Команды установки:

```
apt-get update -y && \\
     apt-get install -y software-properties-common

apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 --recv-keys
3FA7E0328081BFF6A14DA29AA6A19B38D3D831EF

echo "deb <http://download.mono-project.com/repo/ubuntu>
stable-focal main" >
/etc/apt/sources.list.d/mono-official-stable.list && \\
     apt-get update && \\
     apt-get install -y mono-complete && \\
     rm -rf /var/lib/apt/lists/* /tmp/* && \\
     export MONO_GC_PARAMS=max-heap-size=256M
```

Компиляция:

```
csc /unsafe $MOUNT_POINT -out:$COMPILED_FILE_PATH
```

Запуск:

```
mono $MOUNT_POINT > $RESULT_LOCATION 2>&1
```

Обращаем ваше внимание, что вместо переменных вам необходимо вставить свои параметры для компиляции и запуска.

Пример программы:

```
None
using System;
```

```
using System.Linq;
using System.IO;
using System.Text;
using System.Collections;
using System.Collections.Generic;

class Program
{
      static void Main(string[] args)
      {
      string[] inputs;
      inputs = Console.ReadLine().Split(' ');
      int num = int.Parse(inputs[0]);

      for (int i = 0; i < num; i++) {
            inputs = Console.ReadLine().Split(' ');
            var s = int.Parse(inputs[0]);
            Console.WriteLine("{0}", Math.Pow(s, 5));
      }
      }
}
```

Node JS

Операционная система: Ubuntu 20.04

Версия языка Node JS: 1.14

Команды установки:

```
curl -sL <https://deb.nodesource.com/setup_14.x> | bash - && \\
      DEBIAN_FRONTEND=noninteractive apt-get install -y nodejs
npm install prompt decimal.js
```

Запуск:

```
/usr/bin/node $SOLUTION_CODE_PATH/$SOLUTION_CODE_ENTRYPOINT
```

Обращаем ваше внимание, что вместо переменных вам необходимо вставить свои параметры для компиляции и запуска.

Пример программы:

```
var readline = require('readline');
var rl = readline.createInterface({
  input: process.stdin,
```

```
  output: process.stdout,
  terminal: false
});

rl.on('line', function(line){
     console.log(line);
})
```

Примечание: ошибка WA - это могут быть Received empty answer и другие ошибки, которые Node пишет в консоль.

SQL

Операционная система: Ubuntu 20.04

Версия языка SQL: Postgres 12

Команды установки:

```
apt-get install -y postgresql-server-dev-12 gcc python3-dev
musl-dev
```

Удачи!